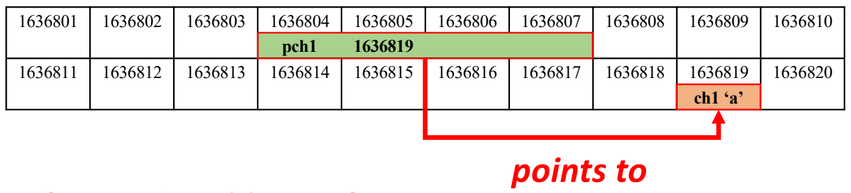
Creative Software Design
2. Lab1
C: Pointers

Dept. of Computer Science

Hanyang University

Pointers

- Variables that store the address of other data
  - The ***type*** provided with a variable is used to specify the encoding for multi-byte data
- Provided with pointer operators
  - &: ***address-of*** operator
  - *: ***indirection*** operator (or ***dereference***)
  - +/-: increment or decrement in a unit of **`sizeof(type)`**



value of **`pch1`** != address of **`pch1`**

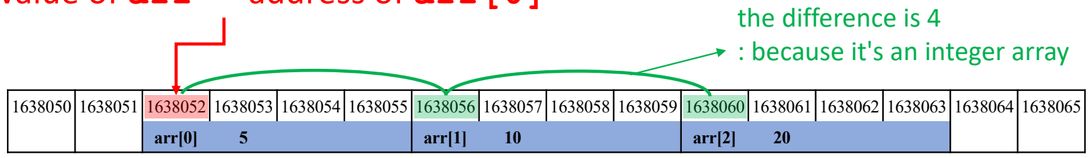
Arrays

- Variables that represent a group of elements
  - All elements MUST be the same type
- All elements are stored in a consecutive memory area
- Variables provide the starting address for the memory

```
int arr[3] = {5, 10, 20};
```

```
arr: 1638052
&arr[0]: 1638052
&arr[1]: 1638056
&arr[2]: 1638060
```

value of **`arr`** == address of **`arr[0]`**



Pointers vs. Arrays

- Common things
  - Both have addresses as values
  - * and [] operators are common for both
- Differences
  - Arrays cannot change the initial address
  - **`sizeof()`** operator returns a different value

1. Iteration over Arrays and Pointers

- Write your own C program
  - Define an array **`arr`** of size 5
  - Iterate over **`arr`** with only pointer operators

```
#include <stdio.h>

int main(void) {
  int arr[5] = {1, 2, 3, 4, 5};

  /* TODO: implement here */

  return 0;
}
```


1. Iteration over Arrays and Pointers (Cnt'd)

- Inspect values in real memory using GDB
  - Compile the Program with Debugging Information

```
$ gcc example01.c -o example01 -g
```

  - Start GDB

```
$ gdb ./example01
```

  - Run the Program

```
(gdb) break main
(gdb) run
(gdb) next
```

  - Inspect variables

```
(gdb) p arr
(gdb) p &arr[0]
(gdb) x/20b arr
```

2. Unsafe Memory Access - 1

- Write your own C program
  - Define an array **`arr`** of size 5
  - *Infinitely* iterate over **`arr`** using index operators (i.e., [])

```
#include <stdio.h>

int main(void) {
  int arr[5] = {1, 2, 3, 4, 5};

  /* TODO: implement here */

  return 0;
}
```


3. Unsafe Memory Access - 2

- Write your own C program
  - Define an array **`arr`** of size 5
  - *Infinitely* iterate over **`arr`** using pointer operators (i.e., ++)

```
#include <stdio.h>

int main(void) {
  int arr[5] = {1, 2, 3, 4, 5};

  /* TODO: implement here */

  return 0;
}
```

4. Passing Arrays to Functions

- Write your own C Program
  - Print the capacity of arbitrary array using **`get_capacity(...)`**

```
#include <stdio.h>

// TODO: complete size_t get_capacity(...);

int main(void) {
  int a[3] = {10, 20, 30};
  int b[7] = {1, 2, 3, 4, 5, 6, 7};

  /* TODO: print the capacity of each array
     using get_capacity(..) */
  printf("%lu, %lu\n", sizeof(a), sizeof(b));
  printf("%lu, %lu\n", get_capacity(a), get_ capacity(b));
  return 0;
}
```


4. Passing Arrays to Functions (Cnt'd)

- Write your own C Program
  - Complete the given **`get_capacity(...)`**

```
#include <stdio.h>

// TODO: complete size_t get_capacity(...);

int main(void) {
  int a[3] = {10, 20, 30};
  int b[7] = {1, 2, 3, 4, 5, 6, 7};

  /* TODO: print the capacity of each array
     using get_capacity(..) */
  printf("%lu, %lu\n", sizeof(a), sizeof(b));
  printf("%lu, %lu\n", get_capacity(a), get_ capacity(b));
  return 0;
}
```

5. *call-by-value* vs. *call-by-reference*

- Guess what will happen in the given source code
  - Compare the results from each function call

```
#include <stdio.h>

void make_2x_int(int a) {
  a = a * 2;
}

void make_2x_arr(int arr[], int len) {
  int i;
  for (i = 0; i < len; i++)
    arr[i] = arr[i] * 2;
}

int main(void) {
  int i = 0, val = 10, arr[5] = {1, 2, 3, 4, 5};
  make_2x_int(val);
  make_2x_arr(arr, 5);
  printf("val: %d\n", val);
  for (i = 0; i < 5; i++)
    printf("arr[%d]: %d\n", i, arr[i]);
  return 0;
}
```